

Rock–Paper–Scissors Game Report

In this homework, I implemented a Rock–Paper–Scissors game using MPI with two processes. Each process generates a random move in every round, and the game continues until one of the processes reaches 5 points. Process 0 is responsible for deciding the winner and printing the results.

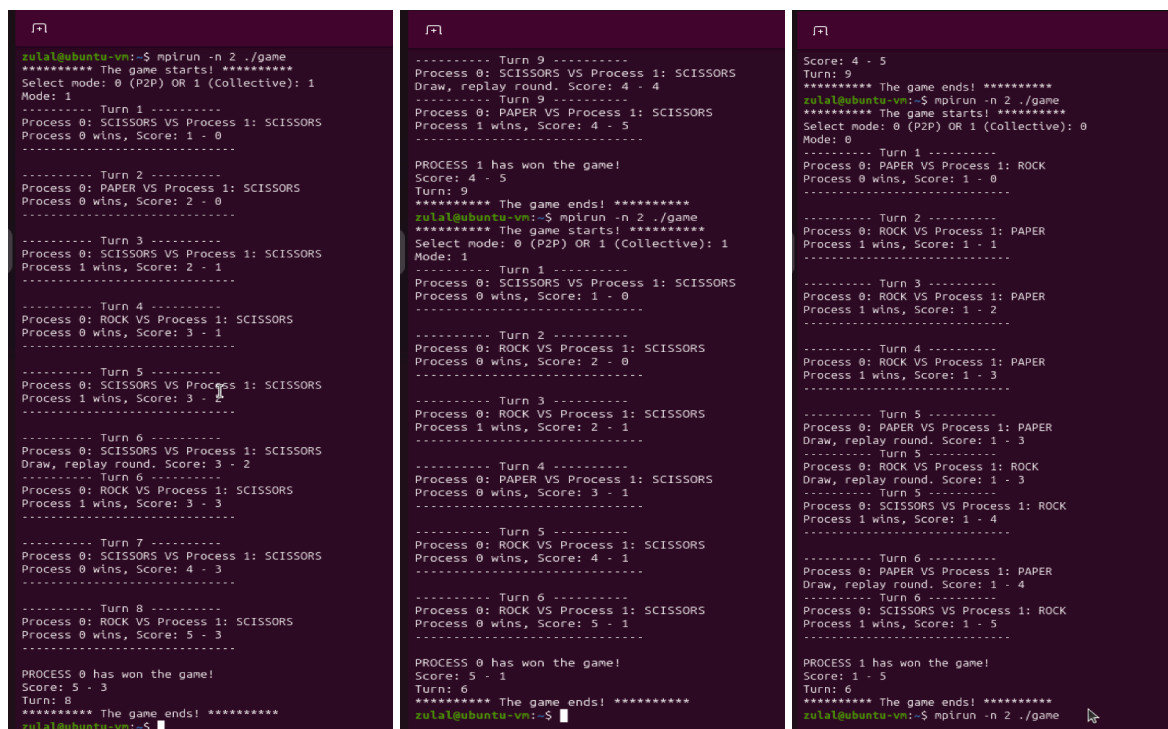
The program was written in C and uses MPI for communication. At each round, both processes generate a random move. Depending on the selected mode, the communication is handled differently.

In point-to-point mode, I used *MPI_Send* and *MPI_Recv* to transfer moves between processes. Process 1 sends its move to Process 0, and Process 0 determines the winner and sends the game status back. In collective mode, I used *MPI_Gather* to collect moves from both processes and *MPI_Bcast* to share the game status. Process 0 collects all moves, determines the result, and broadcasts whether the game is finished.

The most challenging part for me was designing the overall structure of the program. At the beginning, it was not clear how to divide responsibilities between the two processes. Especially deciding which operations should be handled only by Process 0 and which should be handled by Process 1 took some time.

There were also parts in the code where too many nested conditions were used, which made me question whether I was placing some parts in the correct position. In addition, writing and editing the code in a Linux terminal environment made the process slightly more challenging.

The screenshots show three consecutive runs with different results due to randomness.



```
zula@ubuntu-vm:~$ mpirun -n 2 ./game
***** The game starts! *****
Select mode: 0 (P2P) OR 1 (Collective): 1
Mode: 1
----- Turn 1 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Process 0 wins, Score: 1 - 0
-----
----- Turn 2 -----
Process 0: PAPER VS Process 1: SCISSORS
Process 0 wins, Score: 2 - 0
-----
----- Turn 3 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Process 1 wins, Score: 2 - 1
-----
----- Turn 4 -----
Process 0: ROCK VS Process 1: SCISSORS
Process 0 wins, Score: 3 - 1
-----
----- Turn 5 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Process 1 wins, Score: 3 - 2
-----
----- Turn 6 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Draw, replay round. Score: 3 - 2
-----
----- Turn 6 -----
Process 0: ROCK VS Process 1: SCISSORS
Process 1 wins, Score: 3 - 3
-----
----- Turn 7 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Process 0 wins, Score: 4 - 3
-----
----- Turn 8 -----
Process 0: ROCK VS Process 1: SCISSORS
Process 0 wins, Score: 5 - 3
-----
PROCESS 0 has won the game!
Score: 5 - 3
Turn: 8
***** The game ends! *****
zula@ubuntu-vm:~$

----- Turn 9 -----
Process 0: SCISSORS VS Process 1: SCISSORS
Draw, replay round. Score: 4 - 4
-----
----- Turn 9 -----
Process 0: PAPER VS Process 1: SCISSORS
Process 1 wins, Score: 4 - 5
-----
PROCESS 1 has won the game!
Score: 4 - 5
Turn: 9
***** The game ends! *****
zula@ubuntu-vm:~$ mpirun -n 2 ./game
***** The game starts! *****
Select mode: 0 (P2P) OR 1 (Collective): 0
Mode: 0
----- Turn 1 -----
Process 0: PAPER VS Process 1: ROCK
Process 0 wins, Score: 1 - 0
-----
----- Turn 2 -----
Process 0: ROCK VS Process 1: PAPER
Process 1 wins, Score: 1 - 1
-----
----- Turn 3 -----
Process 0: ROCK VS Process 1: PAPER
Process 1 wins, Score: 1 - 2
-----
----- Turn 4 -----
Process 0: ROCK VS Process 1: PAPER
Process 1 wins, Score: 1 - 3
-----
----- Turn 5 -----
Process 0: PAPER VS Process 1: PAPER
Draw, replay round. Score: 1 - 3
-----
----- Turn 5 -----
Process 0: ROCK VS Process 1: ROCK
Draw, replay round. Score: 1 - 3
-----
----- Turn 5 -----
Process 0: SCISSORS VS Process 1: ROCK
Process 1 wins, Score: 1 - 4
-----
----- Turn 6 -----
Process 0: PAPER VS Process 1: PAPER
Draw, replay round. Score: 1 - 4
-----
----- Turn 6 -----
Process 0: SCISSORS VS Process 1: ROCK
Process 1 wins, Score: 1 - 5
-----
PROCESS 1 has won the game!
Score: 1 - 5
Turn: 6
***** The game ends! *****
zula@ubuntu-vm:~$ mpirun -n 2 ./game
```